



2024 PYQ

Ans-1 (a) Binary tree is a hierarchical data structure where each node has at most ~~2~~ two children nodes. ~~Binary~~ Binary tree does not ~~follow~~ follow any specific ordering on ~~its~~ its elements. ~~In~~ In binary search tree elements at ~~the~~ values at left subtree are smaller than values at right subtree.

(b) Final array track[] after performing func() is [0, 50, 30, 40]

(c) A linked list is more suitable for frequent insertions and deletions as it ~~stores~~ stores elements are stored ~~in~~ non contiguously in memory and Insertion and deletion takes $O(1)$ or $O(n)$ time. Whereas array stores elements in contiguous memory and it requires $O(n)$ time in insertion and deletion and its size is fixed and cannot be changed.

(d) A circular queue is considered full when next ^{size} element ~~of~~ of rear is front i.e. $(rear + 1) \% \text{size} = \text{front}$

code

```
bool isFull() {
    if ((rear + 1) % size == front) {
        return true;
    }
    else {
        return false;
    }
}
```



Ans-1) Output for recursion(10, 39) = 1309

Output for recursion(62, -8) = -6028

Ans-2) Two stacks implemented in a single array where one stack grows ~~on~~ from one end and other grows from another end.

~~where~~
The stack 1 with ~~with~~ top1 grow gets incremented when an element gets inserted.

The stack 2 with top2 gets ~~top~~ decremented when an element gets added.

The stack became full when top1 meets top2 which translates to

$$\text{top1} == \text{top2} - 1 \quad \therefore \text{top2} > \text{top1}$$

Ans-2)(a) Implementation of stack :

```
template <class T>
Stack<T>::Stack() {
    top = -1;
}
```

```
template <class T>
bool Stack<T>::isFull() {
    return (top == SIZE - 1);
}
```

```
template <class T>
bool Stack<T>::isEmpty() {
    return (top == -1);
}
```



```
template <class T> void Stack<T>::push(T k) {
    if (isFull()) {
        cout << "Stack Full" << endl;
        return;
    }
    test_stack[++top] = k;
}
```

```
template <class T> T Stack<T>::pop() {
    if (isEmpty()) {
        cout << "Underflow : Stack Empty" << endl;
        return T();
    }
    return test_stack[top--];
}
```

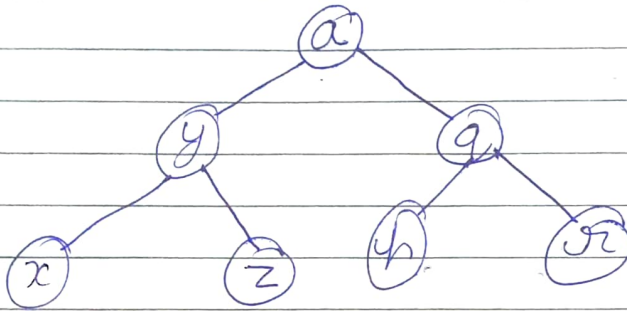
```
template <class T> T Stack<T>::topElement() {
    if (isEmpty()) {
        cout << "Stack is Empty" << endl;
        return T();
    }
    return test_stack[top];
}
```




Ans-2 (b)

In order: x, y, z, a, p, q, r

Pre Order: a, y, x, z, p, q, r

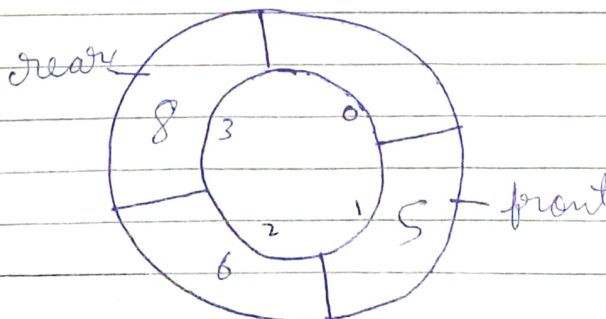


Post Order: x, z, y, p, r, q, a

Ans-(c) Queue of size 4

Operation	Queue	Variables				
Enqueue(4)	<table><tr><td>4</td><td></td><td></td><td></td></tr></table>	4				Front = 0 Rear = 0
4						
dequeue()	<table><tr><td></td><td></td><td></td><td></td></tr></table>					Front = 1 Rear = 0
dequeue()	<table><tr><td></td><td></td><td></td><td></td></tr></table>					Front = 1 Rear = 0
Enqueue(5)	<table><tr><td>5</td><td>5</td><td></td><td></td></tr></table>	5	5			Front = 1 Rear = 1
5	5					
Enqueue(6)	<table><tr><td>5</td><td>5</td><td>6</td><td></td></tr></table>	5	5	6		Front = 1 Rear = 2
5	5	6				
Enqueue(8)	<table><tr><td>5</td><td>5</td><td>6</td><td>8</td></tr></table>	5	5	6	8	Front = 1 Rear = 3
5	5	6	8			

Circular Queue





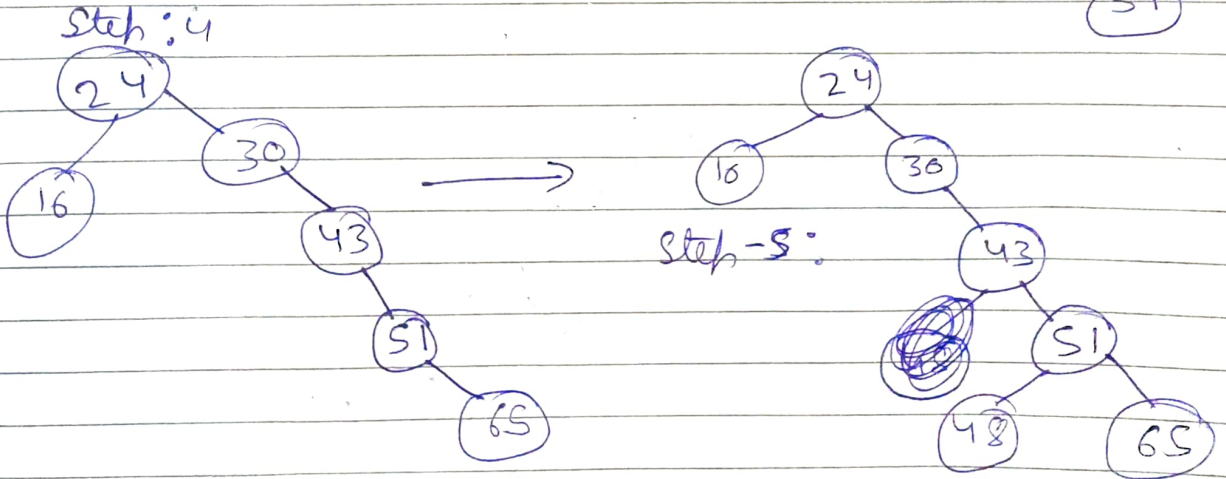
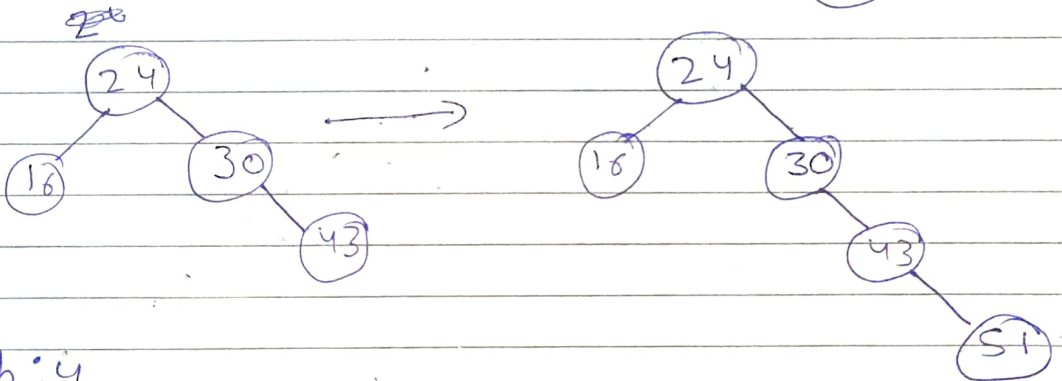
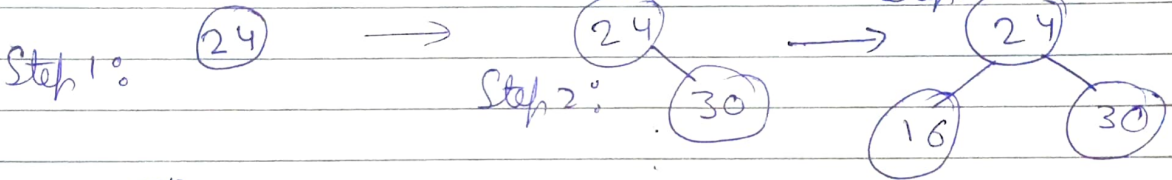
Ans-3 (b) Given linked list:

$8 \rightarrow 12 \rightarrow 91 \rightarrow 13 \rightarrow 42 \rightarrow 5 \rightarrow \text{NULL}$

Output: ~~$5 \rightarrow 12 \rightarrow 91 \rightarrow 13$~~

$5 \rightarrow 42 \rightarrow 13 \rightarrow 91 \rightarrow 12 \rightarrow 8 \rightarrow \text{NULL}$

Ans-3 (c) Given keys: 24, 30, 16, 43, 51, 65, 48, 75, 34, 4

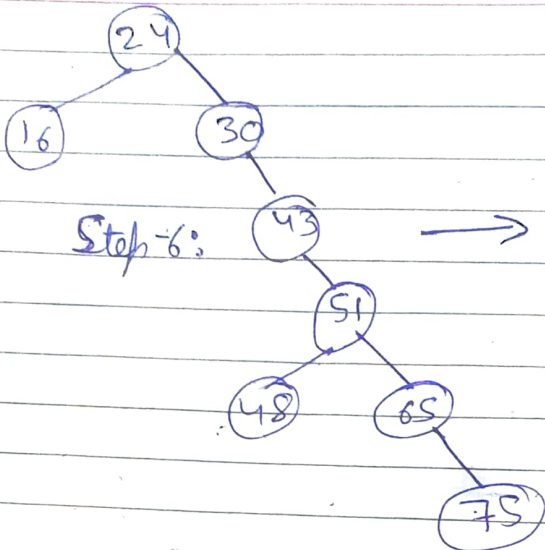




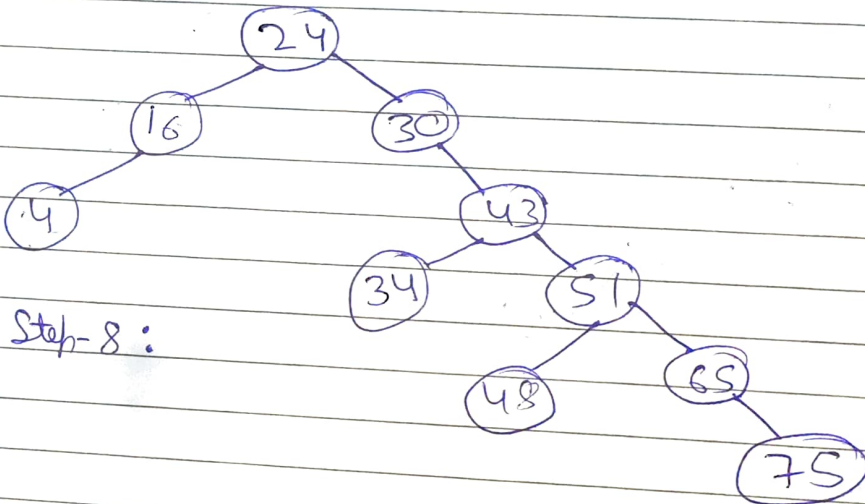
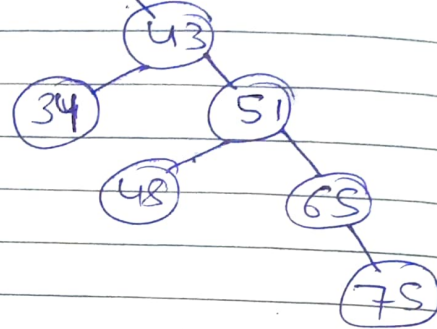
DATE _____

8

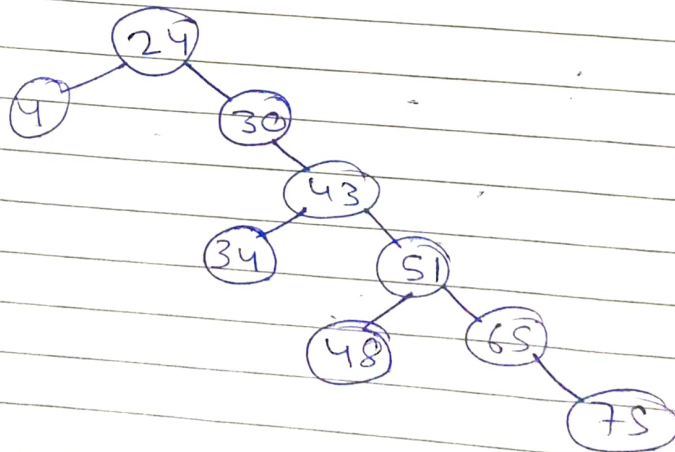
Ans-4 (a)



Step-7:



Deleting -16:
(by copying)





Ans-4 (a) Function to calculate the number of leaves in a binary tree.

```
int countLeaf (Node* r) {
    if (r == nullptr) {
        return 1;
    }
```

```
    return (1 + countLeaf(r->left) + count
        countLeaf(r->right));
```

Ans (b) Stack of size 5

Operation	Stack	Variable					
Push(4)	<table><tr><td>4</td><td></td><td></td><td></td><td></td></tr></table>	4					top = 0
4							
Pop()	<table><tr><td>4</td><td></td><td></td><td></td><td></td></tr></table>	4					top = -1
4							
Pop()	<table><tr><td>4</td><td></td><td></td><td></td><td></td></tr></table>	4					Underflow Error top = -1
4							
Push(5)	<table><tr><td>5</td><td></td><td></td><td></td><td></td></tr></table>	5					top = 0
5							
Push(6)	<table><tr><td>5</td><td>6</td><td></td><td></td><td></td></tr></table>	5	6				top = 1
5	6						
Push(8)	<table><tr><td>5</td><td>6</td><td>8</td><td></td><td></td></tr></table>	5	6	8			top = 2
5	6	8					
Peek()	<table><tr><td>5</td><td>6</td><td>8</td><td></td><td></td></tr></table>	5	6	8			Output : 8 top = 2
5	6	8					
Pop()	<table><tr><td>5</td><td>6</td><td></td><td></td><td></td></tr></table>	5	6				top = 1
5	6						
Pop()	<table><tr><td>5</td><td></td><td></td><td></td><td></td></tr></table>	5					top = 0
5							
Push(90)	<table><tr><td>5</td><td>90</td><td></td><td></td><td></td></tr></table>	5	90				top = 1
5	90						



Ans c) The deque dd

Output:

The deque dd is: 15 20 10 30

4

15

30

20 10 30

20 10

Ans 5 (b)

Code for inorder traversal :

```
void inorder (BSTNode* r)
{
    if (r != NULL) {
        inorder (r -> left);
        cout << r -> data;
        inorder (r -> right);
    }
};
```

Ans-6 (b) L1: 5 → 7 → 9 → 11 → 13 → 15 → NULL

L2: 2 → 4 → 6 → 8 → 10 → 12 → NULL

Output of op-Lists (L1, L2) is

5 → 7 → 9 → 11 → 13 → 15 → 2 → 4 → 6 → 8 → 10 → 12 → NULL



Ans-6 (c) Given Expression: $+ - * 2 2 / 16 8 5$

Symbol	Stack (Top $\rightarrow$ Bottom)	Result
5	5	
8	8, 5	
16	16, 8, 5	
/	16, 8, 5	16 8 /
2	2, 16 8 /, 5	
2	2, 2, 16 8 /, 5	
*	2, 2, 16 8 /, 5	2 2 *
-	2, 2, 16 8 /, 5	2 2 *
	2 2 *, 16 8 /, 5	2 2 * 16 8 / -
-	2 2 *, 16 8 /, 5	2 2 *
	2 2 *, 16 8 /, 5	2 2 * 16 8 / -
	2 2 * 16 8 / -, 5	
+	2 2 * 16 8 / -, 5	2 2 * 16 8 / - 5 +

Final Expression: $2 2 * 16 8 / - 5 +$

Ans 7 (a) (i) Height of tree = 4

(iii) Breadth first traversal: 8, 3, 10,
1, 6, 14, 4, 7, 13

(ii) Number of internal Nodes: 5

(iv) Number of leaves: 4

(b) (i) $f(m) = m^2 \cdot \log n$

$$m \leq C \cdot m^2 \cdot \log n$$

$$m \leq m^2 \cdot \log n \quad \forall m > 0, C=1$$

$$m = O(m^2)$$



$$(ii) f_1(n) = n^{1.5} + 10^6$$

$$n \leq C \cdot (n^{1.5} + 10^6)$$

$$n \leq n^{1.5} + 10^6 \quad \forall n \geq 0$$

$$\text{Hence, } f_1(n) = O(n^{1.5})$$

$$(iii) f_3(n) = 2^n$$

$$n \leq C \cdot 2^n$$

$$n \leq 2^n \quad \forall n \geq 0$$

$$\text{Hence } f_3(n) = O(2^n)$$

$$(iv) f_4(n) = \frac{n^3}{1000}$$

$$n \leq C \cdot \frac{n^3}{1000}$$

$$n \leq 1000 \cdot \frac{n^3}{1000} \quad \forall n \geq 0$$

$$\text{Hence } f_4(n) = O(n^3)$$

$$(v) f_5(n) = \frac{n(n-1)}{2} = \frac{n^2 - n}{2} = \frac{n^2}{2} - \frac{n}{2}$$

$$n \leq C \cdot \left(\frac{n^2}{2} - \frac{n}{2} \right)$$

$$n \leq 2 \cdot \left(\frac{n^2 - n}{2} \right) \quad \forall n \geq 2$$

$$n = O(n^2)$$

Hence the descending order is:

$$f_3 > f_4 > f_5 > f_2$$



Ans-7 (c) Corrected ~~delete~~ ^{delete} ~~insert~~ ^{code} code:

```
void delete () {  
    if (!empty() && head->next != head) {  
        Node *temp = head->next;  
        head->next = head->next->next;  
        delete temp;  
    } else { if (head->next == head) {  
        Node *temptemp = currhead;  
        head = NULL;  
        delete temp;  
    }  
}
```

~~Corrected~~ Corrected insert() code;

```
void insert (int info) {  
    Node *n = new Node;  
    n->data = info  
    n->info = info;  
  
    if (empty()) {  
        cursor = n;  
        n->next = cursor;  
    }  
    else {  
        n->next = cursor->next;  
        cursor->next = n;  
    }  
}
```



Corrected traverse () code :

```
void traverse() {  
    while (head->next != NULL)  
    {  
        cout << head->info << " ";  
    }
```

```
void traverse() {  
    Node * temp = head;  
    while (temp->next != head) {  
        cout << head->info << " ";  
        temp head = temp->next;  
    }  
}
```